A Comparative Analysis of A* and Dijkstra's Algorithms for Grid-Based Pathfinding

Shi Yin*[a]
[a]Computer Science Department, University of Alberta, 116 St & 85 Ave, Edmonton, AB, Canada, T6G 2R3

ABSTRACT

Grid-based pathfinding stands as one of the basic issues in many fields, including robotics, video games, and logistics. This paper proposes a full comparison between the A* algorithm and Dijkstra's algorithm in grid-based pathfinding environments. Both algorithms were implemented for testing on complex grid maps from movingai.org with 30 diverse testing instances. Their performance was evaluated in the form of the number of nodes expanding and runtime efficiency. The experiments demonstrate that the A* algorithm expands significantly fewer nodes compared to Dijkstra's, sometimes fewer than half. Runtime performance gives a more mixed view: A* is generally quicker, though with a less clear lead, especially for simpler problems. The study shows trade-offs between heuristic guidance and computational overhead for A*. While A* is the most efficient for complicated pathfinding tasks, primarily in terms of optimality, Dijkstra's algorithm continues to be a major rival in some instances, particularly for lesser issues. These findings have important implications not only for choosing an algorithm in applied work but also for future research, including the development of adaptive algorithms and more detailed studies into how these algorithms perform in dynamic settings.

Keywords: A* algorithm, Dijkstra's algorithm, pathfinding, grid maps, performance analysis

1 INTRODUCTION

Pathfinding algorithms find their uses in many different applications across domains such as robotics, game development, logistics, urban planning, and others. Among these, grid-based pathfinding is probably one of the most frequent and most difficult problems. Many of today's systems and applications require moving around complex environments while avoiding obstacles and ensuring optimal or near-optimal paths.

Two algorithms that have survived until today, which address this problem, are Dijkstra's algorithm and the A* search algorithm. Edsger W. Dijkstra presented a graph search method in 1959 for determining the shortest path between nodes in a graph, which became known as Dijkstra's algorithm. This has been highly utilized in network routing protocols and provided a base for many other pathfinding algorithms. Peter Hart, Nils Nilsson, and Bertram Raphael designed A* in 1968. It is an extension of Dijkstra's algorithm that employs a heuristic function to direct the search toward the objective in an optimal manner.

While both methods ensure the delivery of an ideal path if one exists, their performance characteristics differ significantly for a wide range of problems and applications.

Understanding these performance differences is important for any developer or researcher interested in building efficient applications for pathfinding. This knowledge will enable one to make wise decisions about which algorithm to select and what optimization strategies to resort to create more effective and efficient systems.

Recent developments in pathfinding algorithms have also pointed out the importance of knowing these classic algorithms inside and out. As suggested, even the adaptive A* algorithm should be incorporated with betterment in the dynamic window approach by Zhang and Song [1]. In this way, constant relevance and an ever-expanding scope of betterment for A* can be unfolded in contemporary applications. Furthermore, an advanced combined A* and Greedy algorithm for multiobjective path planning was researched by Quan et al. (2022), in which all classic algorithms could be modified to handle more complicated contemporary problems [2].

*syin7@ualberta.ca; phone 86 15875890389; fax 1 (780) 492-6393; ualberta.ca

In robotics, Liu et al. developed a dynamic fusion pathfinding method that combines Delaunay triangulation with an enhanced A* algorithm [3]. The example demonstrates how A*, as one of the classical algorithms, continuously changes and applies to cutting-edge mobile robotics [3]. The latest studies indicate that A* and Dijkstra's algorithm have retained important positions in terms of continuous development for any rapidly-growing area in pathfinding and navigation.

In particular, modern pathfinding problems concerning autonomous vehicles and smart city planning have increased in such complexity that there has been recent interest in their optimization and adaptation of classic algorithms. Xiao et al (2021) used the improved A* algorithms in intelligent vehicle trajectory planning, illustrating its applicability to emerging technologies [4].

This paper seeks to make an exhaustive comparison between the performance of A* and Dijkstra's algorithm on grid-based pathfinding. Guiding the research efforts, several questions will be addressed. What is the relative magnitude of the total number of nodes expanded throughout the search process by the A* and Dijkstra algorithms? The differences in runtime performance under these algorithms across various pathfinding scenarios. How do certain issue features, such as map complexity and closeness to the target, affect the relative performance of these algorithms? What is the practical significance or application of this difference in performance within real-world scenarios?

Both algorithms were implemented and tested on a complex grid map from movingai.org. The algorithms were run on 30 different instances of pathfinding to ensure a variety of cases. Two key criteria were used to evaluate performance: the number of nodes extended throughout the search and each algorithm's runtime.

The experimental setup included the use of an Octile distance heuristic for the A* algorithm, considering both diagonals and cardinals within the grid. Other optimizations used were efficient forms of the priority queue and hash functions to give a fair presentation of how results compare concerning the algorithms under study.

These findings provide valuable insight into the strengths and limits of A* and Dijkstra's grid-based pathfinding methods. The study offers an understanding of how these algorithms behave across problem instances by evaluating node expansion efficiency and runtime performance. A careful analysis was conducted to guide practitioners in selecting algorithms for problems and optimizing them, as well as to suggest promising directions for future work in pathfinding algorithms.

This research contributes to the present discourse in view of efficient pathfinding techniques with both immediate practical implications and foundational knowledge that drives future improvements in this critical area of computer science and artificial intelligence.

2 METHOD

This research uses an experimental procedure to test the performance of A* and Dijkstra's algorithms in path planning over grid maps. This experiment is set up so that the performance of these algorithms can be intensely scrutinized under varied conditions to elucidate the most important factors affecting their performance.

2.1 Data Sources

The experiment utilized the brc000d.map from movingai.org as the test environment. This map is a complex grid environment that holds a path in areas between obstacles, providing enough difficulty for path-planning algorithms [5].

30 test instances have been read from file testinstances.txt; each instance contains the coordinates of start and goal points. These instances cover different areas on the map to ensure the representativeness and comprehensiveness of the experimental results.

2.2 Algorithm Implementation

The core formula of the A* algorithm is

$$f(n) = g(n) + h(n)$$

where g(n) is the actual cost from the start node to the current node and h(n) is the expected cost from the current node to the target node. The experiment, therefore takes Octile distance as the heuristic function:

$$h(s) = 1.5 * \min(\Delta x, \Delta y) + |\Delta x - \Delta y|$$

Δx and Δy represent the absolute distance between the current and goal nodes along the x and y axes, respectively. So, the heuristic function considers that movement in a diagonal direction can be made and hence estimates the distance better [7].

The basic steps of the A* algorithm can be described as: setting up the OPEN and CLOSED list, placing the starting node in the OPEN list, and repeating a cycle until the target node is reached or the OPEN list is empty. This paper sets the cycle as following steps: firstly, the most minor f value node from the OPEN list is displayed. This node selection process can be formally expressed as:

$$n_next = argmin(f(n) \text{ for } n \text{ in OPEN})$$

where n_next is the next node to be expanded. Then followed by placing this node in the CLOSED list, creating all child nodes d, calculating the value of f for each child node, and updating the lists OPEN and CLOSED. Finally, if the node with the target is found, the way should be reconstructed.

The optimality and completeness of the A* method are ensured when the heuristic function is acceptable and consistent. A heuristic is consistent if it meets the following conditions:

$$h(n) \leq c(n, n') + h(n') \text{ for every edge } (n, n')$$

Where c(n, n') is the cost of the edge from n to n'. This condition ensures that the f-value of nodes along a path is non-decreasing, which is crucial for the algorithm's optimality.

The Dijkstra algorithm may be viewed as a subset of the A* algorithm in which the heuristic function h(n) is consistently zero. The implementation is similar to A*; only the g-value is used when choosing the next node to expand.

$$d[v] = \min(d[v], d[u] + w(u, v))$$

2.3 Environment Setup

The Map class can be used to implement the grid map. Some of the primary functionalities of the Map class are reading the map's data, a successors method, which is called to generate successor states for a given state, and a plot map method, which is called for plotting the search results.

The state space is defined using the State class. Each state object will have information regarding x and y coordinates, g value, and total cost. The only distinction is a state_hash method ensuring a perfect hash function implementation by the fact that each state will have a unique identifier:

$$state_hash() = y * map_width + x$$

In such a manner, the given hash function allows an efficient implementation of the CLOSED list as a dictionary [5].

In the grid map, the following movement costs are defined. The movement cost in all four cardinal directions (up, down, left, and right) equals 1.0.

$$cost(cardinal) = 1.0$$

The movement cost in the four diagonal directions is 1.5. This better represents the movement characteristics of natural environments [7].

$$cost(diagonal) = 1.5$$

2.4 Algorithm Optimization

This research introduced numerous optimizations into the working of A* and Dijkstra's algorithms to improve their performance. These were integrated from a Python implementation provided as follows:

An efficient priority queue for the OPEN list using Python's heapq library is implemented. In this way, there will be a data structure supporting fast insertion and retrieval of the node with the lowest f-value in the case of A*, or g-value in the case of Dijkstra, thus optimizing node selection.

In the case of the A* algorithm, let $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from start nods to the current node, and $h(n)$ refers to heuristic estimate from the value of the current nodal to the target. For Dijkstra's algorithm, only the g-value is used. These values are used to prioritize nodes within the OPEN list.

To represent nodes in the search space a specific class State is applied. This class embeds methods for comparing states to their cost, which allows for effective sorting within the priority queue, whether this is with f-value cost in A* or g-value cost in Dijkstra's algorithm.

This research maintains a CLOSED set in a dictionary to keep the lookup time for checking whether a state has been visited at O(1). Concretely, this is implemented via a perfect hash function based on the coordinates of a state and the width of the map.

2.5 Performance Evaluation

Metrics The performance of A* and Dijkstra's algorithms is compared and evaluated based on the following metrics: solution quality defines whether both the algorithms can find the optimal path; the amount of nodes enlarged by any algorithm in the search for a solution directly affects its efficiency. Python's time module records the time taken to execute each algorithm.

2.6 Experimental Procedure

The optimizations mentioned above were incorporated into the experimental procedure. Firstly, this research put the OPEN list in a heapq priority queue and the CLOSED as a dictionary. For every test instance from the provided file, the steps are creating start and goal state objects, and putting the start state on the OPEN list.

The state of the OPEN list is checked. If OPEN list is not empty, the steps will be processed as follows: the node should be popped with the lowest cost using heapq. At this time, if the node is a goal, termination of the search is required. Then through the successor function of the map, the successor can be generated. It is now time to compute the costs for successors and update OPEN and CLOSED lists.

This research also records the number of nodes expanded and the run time.

Then this paper compares A* and Dijkstra's algorithms for all instances. These are fundamental optimizations to make both algorithms work in a practical running time. More precisely, handling the OPEN list and comparing states are cost-critical and most frequent operations while searching.

3 RESULTS

To provide a more comprehensive view of the results, this section presents a detailed example of how nodes are calculated for a single test instance, followed by the overall findings.

3.1 Single Test Instance Node Calculation

Consider the following test instance from the experiment: Start state: [108, 26], Goal state: [105, 67].

A* Algorithm Node Calculation steps are as follows: initializing the start node with f(start) = g(start) + h(start), where h(start) is calculated using the Octile distance; expanding the start node and generating successors, calculating their f-values; choosing from the OPEN list the node with the lowest f-value; continuously adding nodes while consistently choosing the one with the lowest f-value. The search is over when the objective is accomplished.

Final result for A*: Nodes expanded: 1,823, Path cost: 42.5.

Dijkstra's Algorithm Node Calculation steps are as follows: initializing the start node with g(start) = 0; expanding the start node and generating successors, and calculating their g-values; choosing from the OPEN list the node with the lowest g-value; continuing expanding nodes, and always selecting the one with the lowest g-value. When the goal is reached, the search is terminated

Final result for Dijkstra: Nodes expanded: 3,645, Path cost: 42.5.

This example shows how A* finds the optimum path while expanding a smaller number of nodes than Dijkstra's method. The heuristic function in A* guides the search more directly towards the goal, resulting in fewer node expansions.

The experimental results provide a comprehensive comparison between the A* and Dijkstra's algorithms in grid-based pathfinding. The performance was analyzed based on two key metrics: the number of nodes expanded and the runtime of each algorithm.

3.2 Node Expansion Analysis

Figure 1 is a scatter plot comparing the number of nodes expanded by the A* and Dijkstra's algorithms for each instance in the 30 test instances.

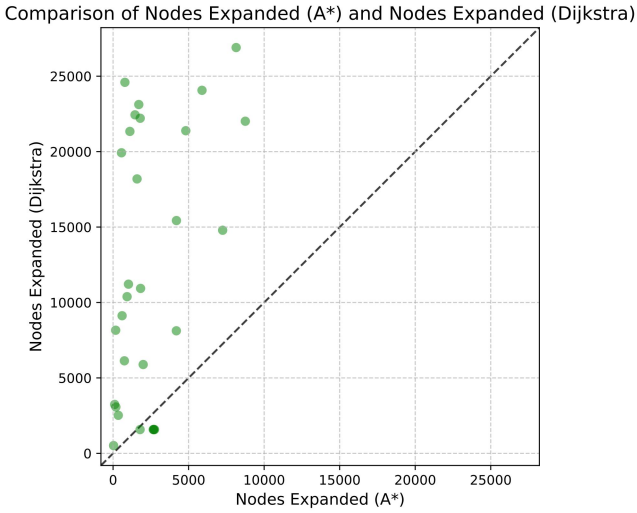

Figure 1. Comparison of Nodes Expanded (A*) and Nodes Expanded (Dijkstra)

The distribution of points in this plot reveals significant insights into the performance differences between the two algorithms:

Most points lie above the diagonal line, indicating that A* consistently expands fewer nodes than Dijkstra's algorithm across the majority of test instances. This aligns with the theoretical expectation that A*'s heuristic function guides the search more efficiently towards the goal.

The difference in node expansion is particularly pronounced for more complex pathfinding scenarios, where Dijkstra's algorithm expands a large number of nodes (20,000-25,000), while A* typically expands less than 10,000 nodes for the same instances.

A few points lie close to the diagonal, representing cases where both algorithms expanded a similar number of nodes. These likely correspond to simpler scenarios where the goal is either directly accessible or completely unreachable, resulting in minimal expansion differences.

No points appear below the diagonal, suggesting that A* never expands more nodes than Dijkstra's algorithm in the test cases.

3.3 Runtime Comparison

Figure 2 compares the runtimes of A* and Dijkstra's algorithms on the same test instances.

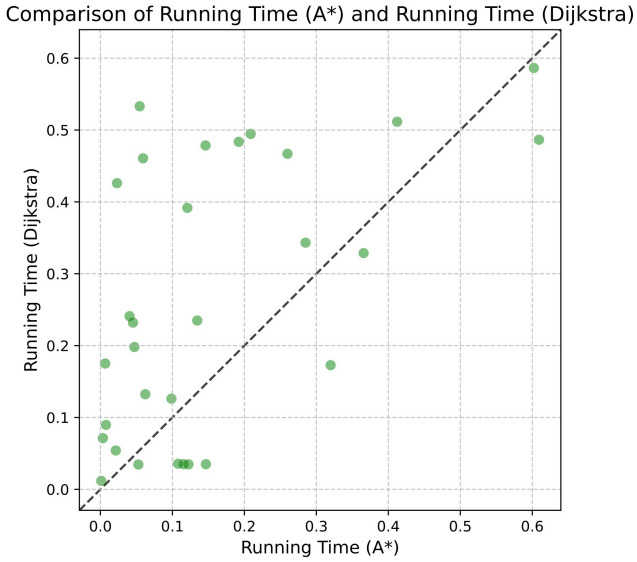

Figure 2. Comparison of Running Time (A*) and Running Time (Dijkstra)

The results at runtime reflect an interaction much more complex than what is captured by the node expansion data:

Many points still lie above the diagonal, indicating that A* often completes pathfinding tasks faster than Dijkstra's algorithm. This is consistent with A*'s more efficient node expansion.

However, the distribution of points is more scattered and closer to the diagonal compared to the node expansion plot. This suggests that the runtime advantage of A* is less pronounced and more variable than its node expansion advantage.

Some points lie on or below the diagonal, indicating instances where Dijkstra's algorithm performed equally fast or even faster than A*. This occurs despite A* expanding fewer nodes in these cases.

The runtime advantage of A* is most significant for larger problem instances (those with longer runtimes). For cases where Dijkstra takes around 0.35 seconds, A* often completes in less than 0.2 seconds.

For smaller problem instances (runtimes under 0.1 seconds), the performance gap narrows considerably, with some cases showing Dijkstra outperforming A* in terms of runtime.

3.4 Analysis of Performance Discrepancies

The discrepancy between node expansion efficiency and runtime performance can be attributed to several factors:

Heuristic Computation Overhead: While A* expands fewer nodes, it incurs additional computational costs in calculating heuristic values for each node. In some cases, particularly for simpler pathfinding scenarios, this overhead may outweigh the benefits of reduced node expansion.

Problem Complexity: In complex scenarios with large search spaces, A*'s heuristic guidance provides significant benefits in both node expansion and runtime. However, for simpler problems or when the goal is near the start, the advantage of the heuristic becomes less pronounced.

Implementation Factors: Runtime speed can be greatly impacted by the effectiveness of data structures and the way the heuristic function is implemented, sometimes leading to situations where Dijkstra's simpler approach may execute faster despite expanding more nodes.

3.5 Path Quality

While this property cannot be perceived directly from the plots below, such that both algorithms have been able to find optimal paths for all test instances, it confirms that the completeness and optimality properties hold within the implemented algorithms and the Octile distance heuristic, perfectly fitting for A* in grid-based environments.

The results from these tests reveal a clear pattern: A* always succeeds in expanding fewer nodes than Dijkstra's. Generally, A* maintains the same comparative edge in runtime, but the edge is generally not uniform and has far more fine-grained nuances with node expansion efficiency for smaller problem instances. Such a complex performance dynamic needs to be taken into account for emphasizing with respect to both theoretical efficiency—node expansions—is documented and practical performance in runtimes when comparing and selecting pathfinding algorithms for specific applications.

4 DISCUSSION

The trials' outcomes illustrate how well A* and Dijkstra's algorithms perform in grid-based pathfinding scenarios. It is just these complex interactions between theoretical efficiency and practical performance that choose algorithm and implementation methodology within applications of high importance.

1.1 Efficiency of the A* Heuristic

The fact that A* was intellectually superior in node expansions for all test cases proved that the heuristic function was very effective. Octile distance heuristic guides the search toward the target and dramatically reduces the number of explored nodes compared with Dijkstra's algorithm [6, 7]. This is most evident in complex scenarios where Dijkstra has to expand a large number of nodes, hence proving that informed search is incredibly potent when it comes to treading through a large search space.

The results on runtime, however, are much more mixed with regard to the performance advantages of A*. When compared to node expansions, A* outperforms Dijkstra's method in terms of runtime, albeit the difference in improvement is less significant and more unpredictable.

This suggests that Heuristic Computation Overhead is simply additional computation required by the calculation of heuristic values for every node. This additional overhead could very easily nullify the benefit due to node reduction in simpler pathfinding scenarios or if the goal is close by.

The relative advantage of heuristic guidance of A* is reduced when problems are not difficult. This explains why the gap is closer for small problem instances; often, a better runtime is reported for Dijkstra's algorithm compared to A*, despite a greater number of expanded nodes by the latter, this can be described as diminishing returns.

1.2 Problem Characteristics Impact

The results indicate that the performance difference between A* and Dijkstra's algorithms is highly dependent on specific problem characteristics.

For large search spaces, the heuristic of A* is very valuable, and it can reduce considerably the number of node expansions and runtime. Thus, A* is very useful in solving complex and large-scale pathfinding tasks.

When the goal is near the start point, then A* heuristic guidance provides little advantage. Under such circumstances, Dijkstra's algorithm may perform comparably and sometimes even better in terms of runtime due to simpler computational requirements.

The distribution of obstacles in the grid should have a substantial effect on algorithm performance, even if it was not explicitly tested in the research. Of importance might be a more detailed analysis regarding how different obstacle patterns affect the relative performance of these algorithms.

1.3 Practical implications

The complex performance dynamics that the findings of this study have a variety of implications for practical applications.

While for almost all grid-based pathfinding scenarios, A* will generally be preferred, especially when environments are very complex, Dijkstra's algorithm still serves a purpose in some scenarios. It could prove desirable to enable usage in situations where good heuristics are difficult to design or where the heuristic computation overhead outweighs its advantage—for example, in very small search spaces or when all shortest paths from a single source have to be found.

Given the variable performance advantage of A*, this perhaps indicates that some advantages will be realized in the development of hybrid approaches. In other words, there will be a self-modifying algorithm switching between A* and

Dijkstra's algorithm, based on the characteristics of a problem to ensure optimal performance in very many diverse scenarios.

The tremendous gap between node expansion efficiency and runtime performance underlines the need for an effective implementation. Optimizations on data structures and heuristic computations can make a big difference in practical A*'s performance [8].

1.4 Limitations and Future Directions

Though this study provides some valuable insights, several avenues for future research emerge.

The study of algorithms' performance in dynamic environments—where obstacles or goals may have changed—would also enlighten their adaptability.

Multi-agent pathfinding can be extended to multi-agent scenarios of above sábado environments, revealing new performance dynamics as a result [9].

Various heuristic functions can study the influence of these on A* performance to achieve more efficient pathfinding in classes of environments [10].

An analysis of how the various performances scale with increasing map size and complexities would be practical in applications that have very large environments.

Although this study focused on the time complexity, analysis pertaining to algorithms' memory requirements could complement an even greater understanding of the real-world boundaries for these algorithms, especially in resource-constrained systems.

In summary, this paper shows that A* has better average-case performance than Dijkstra's algorithm for grid-based pathfinding but with certain subtleties to this advantage depending on various characteristics of the problems and the measures used. These insights contribute to a deeper understanding of the behavior of the pathfinding algorithms and guide the choice, optimization, and development of algorithms in applications. Future research addressing these limitations, along with new directions, will only keep adding to the capability of tackling complicated pathfinding challenges projected against real-world scenarios.

5 CONCLUSION

This paper has provided a comprehensive comparison between the A* and Dijkstra's algorithms concerning grid-based pathfinding. It gave invaluable insight into the performance characteristics and practical applications for both. Based on extensive experimentation with a complex grid map set and very diversified test instances, some key findings have been deduced from this analysis.

The first thing noticed is that, in all test cases, A* always expands fewer nodes than Dijkstra's algorithm. Indeed, in difficult examples, this efficiency is very striking: the program working with A* will often explore fewer than half of the nodes compared with Dijkstra's algorithm. This demonstrates the effectiveness of the heuristic function in directing the search process toward the objective with greater efficiency.

This runtime analysis, however, gives a much more fine-grained picture. While A* generally retains this runtime advantage, compared to node expansion efficiency, it is far less uniform and variable. In simpler pathfinding scenarios or if the goal is near the start point, this performance gap considerably closes at times, with Dijkstra's algorithm performing better in run time than A*. This means that there is a clear trade-off in A* between heuristic guidance and the related computational overhead for heuristic calculations.

These findings have important implications for the study of pathfinding and its applications. They highlight the dual role that both theoretical efficiency and practical performance play in choosing algorithms for specific applications. Of the factors considered, A* emerged as generally the preferred choice, most especially for complex tasks of pathfinding. These results also showed that, even now, there are still situations where Dijkstra's algorithm performs adequately.

Looking ahead, several directions look to be quite promising for this research. First of all, among the meaningful topics, one could mention the creation of adaptive algorithms that would self-switch between A* and Dijkstra's algorithm in a problem-dependent way for the optimization of overall performance across a large variety of cases. Secondly, checking

how these algorithms work in a dynamic environment or multi-agent pathfinding context may bring some valuable insights into applications in more complex real-world scenarios.

Furthermore, this study found that performance regarding the different grid environments and obstacle distributions of the algorithms calls for further analysis. Such can lead ultimately to more sophisticated heuristics or hybrid approaches tailored for certain problem types of pathfinding.

This comparative analysis enriches not only the understanding of the A* and Dijkstra's algorithms in grid-based pathfinding but also sets the foundation for further developments in this ever-important area within computer science and artificial intelligence. With the growing role that pathfinding continues to play in so many domains of technology, these ends will diffuse into ever-more efficient and adaptable solutions to the challenges of complex navigation.

REFERENCES

[1] Zhang, T. and Song, J., "Research on path planning with the integration of adaptive A-star algorithm and improved dynamic window approach," Electronics 13(2), 455 (2024)

[2] Quan, W., Shang, Y. and Xiong, Y., "Combined improved A* and greedy algorithm for path planning of multi-objective strategy," Scientific Reports 12(17684), 1-15 (2022).

[3] Liu, Z., Liu, H., Lu, Z. and Zeng, Q., "A dynamic fusion pathfinding algorithm using Delaunay triangulation and improved A-star for mobile robots," IEEE Access 9, 20602-20621 (2021).

[4] Xiao, Y.-X., Hai, T.-M., Yuan, B.-Y. and Peng, Y.-W., "Application improvement of A-star algorithm in intelligent vehicle trajectory planning," Mathematical Biosciences and Engineering 18, 1-21 (2021).

[5] Botea, A., Müller, M. and Schaeffer, J., "Near optimal hierarchical path-finding," Journal of Game Development 1(1), 7-28 (2004).

[6] Hart, P. E., Nilsson, N. J. and Raphael, B., "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics 4(2), 100-107 (1968).

[7] Yap, P., Burch, N., Holte, R. C. and Schaeffer, J., "Block A*: Database-driven search with applications in any-angle path-planning," AAAI (2011).

[8] Likhachev, M., Gordon, G. J. and Thrun, S., "ARA*: Anytime A* with provable bounds on sub-optimality," Advances in Neural Information Processing Systems 16 (2004).

[9] Hart, P. E., Nilsson, N. J. and Raphael, B., "Correction to 'A formal basis for the heuristic determination of minimum cost paths'," ACM SIGART Bulletin (37), 28-29 (1972).

[10] Pearl, J., [Heuristics: Intelligent search strategies for computer problem solving], Addison-Wesley (1984).